20 SINGLE-THREADED vs MULTI-THREADED — AND HOW TO GET MULTITHREADING IN NODE/NEST

Node.js (and therefore NestJS, which runs on Node) executes YOUR JavaScript/TypeScript code on exactly ONE thread — the "event loop" thread. This surprises people coming from Java/C#/Python-threading backgrounds, where a web server typically spins up a new OS thread per request.

WHAT "SINGLE-THREADED" ACTUALLY MEANS HERE

```
Traditional multi-threaded server (e.g. Java/Spring):
  Request 1 -> Thread A  \
  Request 2 -> Thread B   } all running literally AT THE SAME TIME,
  Request 3 -> Thread C  /  on different CPU cores
```

Node.js / NestJS:

```
Request 1 -> \
Request 2 ->  } all handled by ONE thread, taking turns via the
Request 3 -> /   EVENT LOOP — never two pieces of YOUR code
                  running simultaneously
```

Node is NOT single-threaded in every sense, though — this is the part that's commonly misunderstood:

- YOUR application code (route handlers, service methods, everything you write) runs on ONE thread.
- Node's underlying engine (libuv) DOES use a background thread pool internally for things like file I/O, DNS lookups, and some crypto — but that's invisible plumbing, not something your code directly controls or benefits from for CPU-bound work.

WHY NODE CHOSE THIS MODEL

For I/O-bound work (which is MOST of what a typical HTTP API does — waiting on a database query, an external API call, reading a file), a single thread with non-blocking, event-driven I/O is actually very efficient: while request A is WAITING on the database, the thread isn't sitting idle — it picks up request B, C, D in the meantime. No thread context-switching overhead, no locks, no race conditions on shared memory, because there's only ever one thread touching your objects.

```
async findOne(id: number) {
  const task = await this.taskRepository.findOneBy({ id }); // <-- while
  // WAITING here for the DB, the event loop thread is free to handle
  // OTHER requests. This is why Node handles many concurrent I/O-bound
  // requests well with just one thread.
  return task;
}
```

THE PROBLEM — CPU-BOUND (SYNCHRONOUS) WORK BLOCKS EVERYTHING

Single-threaded only works well if nothing HOGS that one thread. A slow, synchronous, CPU-heavy calculation blocks the event loop — and since there's only one thread, EVERY other request, for EVERY other user, freezes until it finishes.

```
// task.service.ts — a deliberately bad example
@Injectable()
export class TaskService {
  // synchronous, CPU-heavy — BLOCKS the entire server
  slowFibonacci(n: number): number {
    if (n <= 1) return n;
    return this.slowFibonacci(n - 1) + this.slowFibonacci(n - 2); // no
    // await anywhere — this is pure synchronous CPU work
  }
}

@Get('fib/:n')
getFib(@Param('n', ParseIntPipe) n: number) {
  return this.taskService.slowFibonacci(n); // n=45 -> seconds of 100%
  // CPU use, and during those seconds NO OTHER REQUEST to this
  // server — from ANY client — gets handled. GET /task, GET /user,
  // everything queues up behind this one calculation.
}
```

This is the single biggest Node.js production gotcha: `await` and Promises make I/O-bound work non-blocking, but they do NOTHING to help CPU-bound work — a `for` loop or heavy synchronous computation blocks the thread no matter how many `async` / `await` keywords surround it.

HOW TO ACTUALLY GET MULTITHREADING/PARALLELISM IN NODE

There's no way to make ONE Node process's event loop itself run your code on multiple threads simultaneously — but there are real ways to use MULTIPLE threads or processes for actual parallel CPU work:

1. worker_threads — TRUE MULTITHREADING WITHIN ONE PROCESS

Built into Node itself. Spins up a separate thread with its OWN V8 instance and its OWN event loop, running in parallel with the main thread. Communicate via message-passing (not shared memory by default), so no manual locking needed.

```
// fib.worker.ts — runs ON THE WORKER THREAD
import { parentPort, workerData } from 'worker_threads';

function fib(n: number): number {
  return n <= 1 ? n : fib(n - 1) + fib(n - 2);
}

parentPort?.postMessage(fib(workerData.n));

// task.service.ts — runs ON THE MAIN THREAD, offloads the work
import { Worker } from 'worker_threads';

@Injectable()
```

```ts
export class TaskService {
  runFibInWorker(n: number): Promise<number> {
    return new Promise((resolve, reject) => {
      const worker = new Worker('./dist/fib.worker.js', { workerData: { n } });
      worker.once('message', resolve);
      worker.once('error', reject);
    });
  }
}
```

Now the main thread's event loop is FREE while the Fibonacci calculation runs on a different thread — other requests keep being served normally. Best fit: genuinely CPU-heavy work (image processing, complex calculations, encryption) inside an otherwise I/O-bound app.

Nest-specific note: there's no special "NestJS worker_threads" API — you use Node's own worker_threads module directly inside a service, as shown above. For a cleaner setup, packages like `piscina` or `@nestjs/bullmq` (job queues) wrap this pattern for you.

2. cluster / PM2 CLUSTER MODE — MULTIPLE PROCESSES, ONE PORT

Instead of one thread doing more, run SEVERAL SEPARATE Node processes (each with its own single-threaded event loop), all listening on the same port — the OS/cluster module load-balances incoming connections across them.

```ts
// cluster wrapper around main.ts (conceptual)
import cluster from 'cluster';
import os from 'os';

if (cluster.isPrimary) {
  const cpuCount = os.cpus().length;
  for (let i = 0; i < cpuCount; i++) cluster.fork(); // one process per CPU core
} else {
  bootstrap(); // each worker process runs the actual Nest app
}
```

In practice, most teams don't hand-roll this — they use PM2:

```
pm2 start dist/main.js -i max   # "-i max" = one process per CPU core
```

This is the standard way to use ALL CPU cores for a Node/Nest HTTP API in production. Each process is still internally single-threaded, but now you have N independent event loops instead of 1, so N requests can be TRULY handled in parallel across cores — great for scaling overall THROUGHPUT of many I/O-bound requests, not for making one single slow calculation finish faster (that specific calculation still blocks whichever ONE process happens to receive it).

3. child_process.fork() — ONE-OFF OFFLOADING TO A SEPARATE PROCESS

Similar idea to worker_threads but a full separate OS process (heavier than a worker thread, but fully isolated — a crash in the child can't take down the parent). Useful for occasionally running a heavy, isolated script (report generation, a batch job) without touching the main API process at all.

```
import { fork } from 'child_process';
const child = fork('./report-generator.js');
child.send({ reportId: 123 });
child.on('message', (result) => { /* ... */ });
```

WHICH ONE TO REACH FOR

```
|  Problem                                    | Solution              |
|---------------------------------------------|-----------------------|
| One request does heavy CPU work, freezing     | worker_threads        |
| everyone else                                 |                        |
| The whole app needs to use more CPU cores     | cluster / PM2 -i max   |
| for overall throughput                         |                        |
| An occasional heavy, isolated batch job       | child_process.fork()  |
| Waiting on a DB/API call (I/O, not CPU)       | Just use async/await   |
|                                               | — no threading needed |
|                                               | at all, this is what  |
|                                               | Node already does well|
```

★ RULE OF THUMB

Don't reach for threading to fix slow I/O — `async` / `await` already handles that efficiently on a single thread (see the findOne() example above). Only reach for worker_threads/cluster/child_process when the bottleneck is genuinely CPU-bound synchronous work that would otherwise freeze the entire event loop for every other user of the app.

See also: theory/21-worker-threads.txt (deep dive on option 1 above), theory/07-dependency-injection.txt, theory/00-project-structure.txt